Stage 0: the environment gate

run of 2026-11-01

This is the complete account of Stage 0 of the experiment plan: the protocol, the run, the results and how to reproduce them. It is written so that a reader who has the paper but not the scripts can see exactly what is done to obtain every controller, every number and every verdict. The paper's Stage 0 subsection summarises this document; §11 says which parts go into the paper's appendix.

Provenance. Every table in this document is generated from a result file by `scripts/stage0_render.py` and names the commit and the job that produced it. The gate was run twice. Run 1, code commit `fc23a2b6effd1c6e3e28a9a37a0c5325384db7db`, Slurm job 2978483 on gx07, 2026-11-01, failed G3 at the bottom anchor and led to one change in the hand-written controller (§9.4); its files are `docs/stage0/stage0_2978483/stage0_2978483.out`, `docs/stage0/stage0_2978483/stage0_results.jsonand` `docs/stage0/stage0_2978483/stage0_rows.json`. Run 2, code commit `2edeba7c173ce738e608d56eb7a575122e4a02bd`, Slurm job 2978644 on gx07 (32 CPUs, 278 s), 2026-11-01, is the run every other table reports; its files are the summary log `docs/stage0/stage0_2978644/stage0_2978644.out`, the result `docs/stage0/stage0_2978644/stage0_results.json`, and every per-batch row in `docs/stage0/stage0_2978644/stage0_rows.json`. All are tracked in git. The held-out task file the references use is identified by the SHA-256 prefix `201d522e9287`. §10 gives the commands.

Terms. A *controller* is a Python function `policy(state)` that returns one action per step; the paper also calls it the policy or the final run. A *batch* is one episode of the crystallizer: one seed fixes its start temperature, its target temperature and its seed loading. The *hidden limit* $\sigma_{\max}$ is the level of supersaturation at which the batch showers and is ruined. A *blind* controller does not know $\sigma_{\max}$; an *informed* controller was tuned for one known value of it; the *oracle* is the controller that rides exactly at $\sigma_{\max}$ and sets the time-optimal reference. *Fast success* means the batch succeeds within 1.25× the oracle's step count for that batch and limit. *Normalised time* is the oracle's steps divided by the controller's own steps, and zero on failure. An *anchor* is one fixed value of $\sigma_{\max}$ at which the informed side is tuned and both sides are reported. A *third* is one of three equal parts of the limit's range, low, middle and high.

1 What Stage 0 is, and what it is not

Stage 0 does not use a language model. No prompt is written, nothing is told to anything, and no text is generated. Every controller in Stage 0 is a short formula written by hand, with between one and ten numerical knobs, and the knobs are set by a simple search on the environment. The stage is CPU only.

Its purpose is to answer one question before any model run is spent: *does knowing the hidden limit buy anything that a controller without it cannot get from the readings of its own batch?* The whole study rests on a yes. The paper's thesis is that a first-turn probe run, which finds the

limit by crossing it on a throwaway batch, is worth credit because the final controller needs the limit and cannot learn it in-episode. If a blind controller could be nearly as fast as an informed one, the probe would be worthless, every training arm would learn to skip it, and the credit assignment the paper proposes would have nothing to assign.

The gate therefore sits before everything else in the ladder, and it also produces the two ends of the scale every model stage is read against: the blind reference and the oracle, on the very batches the model stages are evaluated on (§9.6).

2 The environment, as much as Stage 0 needs

A stirred tank holds one kilogram of solvent with dissolved solute and a known mass of small seed crystals. A jacket cools or heats it. The controller sets the jacket rate once every 20 s step as a fraction of the jacket's limit of 1.5 °C/min, so one step of full cooling lowers the temperature by 0.5 °C; the action is that fraction, a float in $[-1, 1]$, and nothing else. The batch starts saturated at 55–65 °C and must reach a target of 18–25 °C. Cooling raises the supersaturation σ, the fraction by which the dissolved amount exceeds what the liquid can hold at the current temperature; the seeds grow and consume it. The state reports σ as a reading. Success means reaching the target temperature (within 0.25 °C) with $\sigma \leq 0.02$. If the reported σ ever exceeds the hidden limit $\sigma_{\max}$, drawn once per task from $U[0.08, 0.25]$, a shower of new nuclei ruins the batch and the episode ends in failure. The horizon is 600 steps. The reward is -0.3 per step, $+120$ on success and -80 on failure. The full dynamics and constants are in the paper's crystallizer appendix.

The hidden limit appears nowhere except in the shower test. Two batches with different limits produce identical readings under identical actions until one of them showers, so no in-episode estimator exists. The fastest safe recipe holds σ just below $\sigma_{\max}$ and accelerates as the crystals grow (classical programmed cooling). A controller that does not know the limit must hold σ at the bottom of the range to be safe on every batch, and is then slower than the oracle, the more so the higher the batch's limit.

One method of the environment matters for the hand-written side: `peek(action)` returns the σ the state would report after the action, without taking it. It is the same arithmetic as the step itself, so a lookahead built on it is exact. Hand-written controllers may use it, since they stand for the best a controller can do with the dynamics known; a controller the model writes sees only `state`.

3 What a controller is in Stage 0

Five hand-written families, each with knobs $p_k \in [0, 1]$ mapped to a physical range (Table 1). Four of them compute a desired temperature T_{des} for the end of the next step and return the jacket fraction that gets there, clipped to the jacket's limit and never below the target temperature. The fifth, *track*, is the *rider*: the most cooling such that the reported σ after the step stays at or below its level, found by bisection on `peek`, and never below the target. The rider at the hidden limit is the oracle; the rider at the bottom of the range is the cautious blind reference; the track family's one knob is its level. *hybrid* caps a power profile with the rider, so its σ never exceeds the cap.

4 The two sides, the batches and their limits

A batch's hidden limit is a deterministic function of its seed, drawn uniformly within one third of the range: the third is the batch's index in its set modulo three, so every set holds equal

Family	Knobs	Physical parameters	What it does
track	1	level $= 0.02 + 0.33\,p_0 \in [0.02, 0.35]$	The rider: hold the reported σ at the level. Feedback, with the lookahead.
linear	1	rate $= 0.02 + 1.48\,p_0$ °C/min	Cool at one constant rate until the target. Open loop.
power	2	$\tau = 10 + 190\,p_0$ min; $n = 0.5 + 3.5\,p_1$	Follow $T(t) = T_0 - (T_0 - T_{\text{target}})\,\min(1, t/\tau)^n$. Open loop.
hybrid	3	τ, n as above; cap $= 0.02 + 0.33\,p_2$	The power profile, never cooling harder than the rider at the cap allows. Feedback, with the lookahead.
pwl	10	four knot times $p_0..p_3$ on the horizon fraction $t/600$; six levels $p_4..p_9$	Piecewise-linear profile: $T_{\text{des}} = T_{\text{target}} + f(t)\,(T_0 - T_{\text{target}})$ with f linear between knots. Open loop.

Table 1: The five hand-written controller families (`scripts/stage0_gate.py`, `make_controller`). T_0 is the batch's start temperature, remembered at the first call.

numbers of low, middle and high limits. The informed side is tuned and reported at three anchors, $\sigma_{\max} = 0.08$, 0.165 and 0.25, the ends and the middle of the range.

The **blind side** gets one controller per family. Its knobs are tuned on the 48 search batches (seeds 20000 to 20047), whose limits vary by third, so the single controller must do well across all of them without ever being told which limit a batch has. Nothing in its inputs depends on $\sigma_{\max}$.

The **informed side** gets one controller per family per anchor. Its knobs are tuned on the same search batches with every limit set to the anchor. It "knows" the limit only in this sense: it was tuned for it. It reads the same state and has the same lookahead as the blind side.

Both sides use the same families, the same search budget, the same search seeds, the same scoring batches and the same objective, the mean environment reward, which is what the later reinforcement learning maximises. The gate therefore tunes 5 blind controllers and $5 \times 3 = 15$ informed ones.

5 How the knobs are tuned

Random sampling followed by hill climbing, identical for every family and both sides, driven by one random number generator seeded with 0:

1. Draw 300 knob vectors uniformly in the unit cube of the family's dimension.
2. Run each on the search batches and record the mean reward. A controller that raises an exception on a batch scores the runner's error floor, -300, on that batch.
3. Keep the best 6.
4. For 120 rounds: nudge each survivor by Gaussian noise with a scale that shrinks from 0.16 to 0.01 over the rounds, clip to the cube, evaluate the nudged vector on the same batches, and keep it in place of its parent if it scores higher.
5. Return the best survivor.

That is 1020 candidate evaluations per controller, each over 48 batches.

6 How a controller is scored

Tuned controllers are scored on 200 fresh batches (seeds 30000 to 30199) that the search never saw, with limits by third as above, or with every limit set to an anchor. Three numbers are

averaged.

- **Success**: reached the target temperature with $\sigma \leq 0.02$, no shower, no seed loss, within 600 steps.
- **Fast success**: success and steps $\leq 1.25\times$ the oracle's steps on that batch at that limit.
- **Normalised time**: oracle steps divided by own steps; zero on failure.

The oracle is the rider at the limit, computed once per (batch, limit) pair, and it is the same oracle whose step counts the task files store, so fast success means the same thing here and in every model stage. Every batch is drawn as the runner draws it: construct the environment, then reset once. Fast success is the headline number because raw success is uninformative: a cautious blind controller succeeds on every batch by being slow.

7 The three gates

All three were fixed before the run. G1 and G2 come from the search; G3 from the hand-written two-turn solution of §8, through the real execution path.

Gate	Condition	What it checks
G1	informed fast success $\geq 90\%$ at every anchor	The task is doable when the limit is known, and the search finds the controller that rides it.
G2	the best blind controller's fast success over the range is at least 40 points below the informed side's mean over the anchors	The limit matters: not knowing it costs speed that no blind controller recovers.
G3	the hand-written probe run plus controller, from the brief alone, through the sandbox, reaches $\geq$ 95% fast success at every anchor with no probe failure	The gap is closable by probing, with exactly what the model is told.

Table 2: The pre-registered gates (`GATES` in `scripts/stage0_gate.py`).

G1 without G2 would mean the environment is easy without the limit, so the probe is pointless. G1 and G2 without G3 would mean the limit matters but cannot be found with what the model is given, so the probe is pointless for a different reason. All three together say the probe is the way through.

8 The hand-written probe run and controller

This is the one place in Stage 0 where something has the shape of what the model is later asked to do, except that a person wrote it, using only what the brief offers: the probe-run object `env` with `reset`, `step`, the end flag and `failure_reason`, and the σ reading; the controller sees only `state` and may keep memory. No solubility curve, no concentration arithmetic, no lookahead. Both go through the real execution path: the probe source through the runner's turn executor, which runs it in a sandboxed subprocess against a probe-run environment that has the same hidden limit as the scored batch but a different batch and exposes nothing else; its printed bracket pasted into the controller; the controller scored with the same function the trainer uses, on the scoring batch.

Why the probe has to cross the limit. The limit shows up only as the shower. A batch that never showers says nothing about it. So the probe cools until the batch showers: the highest

reading seen before the shower is a lower bound on $\sigma_{\max}$ and the reading at the shower is an upper bound. Full cooling raises σ by about 0.01 per step, as the quench logs in the brief show, so one batch brackets the limit to about the rise of one step.

The probe (three passes). Pass one cools at the jacket limit until the shower. Passes two and three reset, go quickly to the lower bound, then raise the setpoint in eighths of the bracket until the shower, using a cautious guess of how much σ rises per step of full cooling to turn a setpoint into an action. It prints the bracket. The listing is generated from `scripts/stage0_solution.py`.

```
# Probe run: find the level of sigma at which this crystallizer showers, by crossing it.
lo, hi = 0.0, 0.40    # sigma_max is above lo (a reading seen safe) and below hi (a reading at a shower)
G = 0.016             # cautious guess at how much sigma rises per full-cooling step
for npass in range(3):
    s, _ = env.reset()
    if npass == 0:
        while True:         # first batch: cool at the jacket limit until the shower
            s, r, done, _, _ = env.step(-1.0)
            if done:
                break
            lo = max(lo, s["sigma"])
        if env.failure_reason == "shower":
            hi = min(hi, s["sigma"])
        continue
    target, inc = lo, (hi - lo) / 8.0      # later batches: go to the last safe level, then creep up
    while True:
        a = max(-1.0, min(0.0, -(target - s["sigma"]) / G))
        s, r, done, _, _ = env.step(a)
        if done:
            break
        lo = max(lo, s["sigma"])
        if s["sigma"] >= target - inc / 2:
            target = min(target + inc, hi)
    if env.failure_reason == "shower":
        hi = min(hi, s["sigma"])
print(f"SIGMA_MAX_LOW={lo:.5f} SIGMA_MAX_HIGH={hi:.5f}")
```

The controller. It holds the reading 0.003 under the lower bound the probe printed. It needs memory, because the crystals consume supersaturation between steps and a controller that ignores that either undershoots or showers: it estimates each step how much the crystals ate, as what cooling should have added minus what the reading shows, smooths that estimate, and adds it back to the cooling demand. It never overshoots the target temperature.

```
SETPOINT = max(<SIGMA_MAX_LOW>, 0.08) - 0.003   # the limit is at least 0.08
G = 0.014                       # about how much sigma rises per full-cooling step
mem = {"sigma_prev": None, "a_prev": 0.0, "eaten": 0.0}
def policy(state):
    sigma, T, T_target = state["sigma"], state["T"], state["T_target"]
    if mem["sigma_prev"] is not None:
        # what the crystals ate last step: what cooling should have added, minus what we got
        eaten = mem["sigma_prev"] - mem["a_prev"] * G - sigma
        mem["eaten"] = 0.5 * mem["eaten"] + 0.5 * max(0.0, eaten)
    if T <= T_target:
        a = 0.0
    else:
        a = max(-1.0, min(0.0, -(SETPOINT - sigma + mem["eaten"]) / G))
        a = max(a, -(T - T_target) / 0.5)      # do not overshoot the target temperature
    mem["sigma_prev"], mem["a_prev"] = sigma, a
    return a
```

How much probing is needed. Four probe variants are run over the range, each followed by the same controller: the three passes above, which is the gate's variant; one pass at the jacket limit, which stops at the first shower; and one pass that raises its setpoint by a fixed increment every step until the shower, at 0.005 and at 0.001 per step. The question is whether one batch is enough, and whether a slow creep, which at high limits may never cross the limit because the growing crystals consume supersaturation faster than the creep adds it, can still bracket it.

9 Results

9.1 The blind side

Family	Tuned to	Reward	Success	Fast	Norm. time	Failures
track	level 0.1016	44.5	88.5%	10.5%	0.556	23 shower
hybrid	τ 52 min, n 0.50, cap 0.1015	44.2	88.5%	10.5%	0.552	23 shower
pwl	10 knobs	17.3	90.0%	0.0%	0.371	20 shower
power	τ 93 min, n 1.60	7.8	82.0%	0.0%	0.333	36 shower
linear	0.447 °C/min	−2.9	68.5%	0.0%	0.283	63 shower

Table 3: Blind side: one controller per family, tuned across the range, scored on the 200 fresh batches. Code commit `2edeba7`, Slurm job 2978644, from `docs/stage0/stage0_2978644/stage0_results.json`.

Controller	Batches	Reward	Success	Fast	Norm. time	Failures
cautious (rides at 0.08)	low third	40.1	100.0%	22.4%	0.689	none
	middle third	37.5	100.0%	0.0%	0.443	none
	high third	38.5	100.0%	0.0%	0.373	none
	all	38.7	100.0%	7.5%	0.502	none
searched blind (track, level 0.1016)	low third	12.7	65.7%	31.3%	0.542	23 shower
	middle third	60.2	100.0%	0.0%	0.610	none
	high third	60.9	100.0%	0.0%	0.515	none
	all	44.5	88.5%	10.5%	0.556	23 shower

Table 4: The two blind references on the same fresh batches, by third of the limit's range: the cautious controller, which rides at the bottom of the range and is safe on every batch, and the searched blind controller, which maximises the reward. Code commit `2edeba7`, Slurm job 2978644, from `docs/stage0/stage0_2978644/stage0_results.json`.

Reading. Only the two families with the lookahead, *track* and *hybrid*, come close; the three open-loop profiles cannot be both safe and fast without feedback on the reading. The searched blind controller settles *above* the bottom of the range (Table 3, column "tuned to"): under this reward it pays to ride faster and lose the batches whose limit lies below the level, so it showers on every one of those and succeeds on the rest, and is still fast on only one batch in ten. The cautious controller never fails and is fast only on the batches whose limit sits near the bottom, taking about twice the oracle's time over the range (Table 4). Two consequences for the model stages. The no-failure blind reference is the cautious controller. The ceiling of an arm trained on reward without a probe (arm C) is the searched blind controller, which gambles; whether a trained controller gambles is therefore a reading in its own right.

9.2 The informed side

Reading. With the limit known, the two feedback families ride it and are fast on every batch at every anchor, and their mean reward at each anchor is the oracle's. The open-loop profiles cannot ride the limit even when tuned for it: the safe cooling rate accelerates as the crystals grow, and a fixed profile either showers early or crawls. The piecewise-linear profile with ten knobs gets close only at the top anchor, where the limit is loose. G1 therefore says two things: the task is doable when the limit is known, and doing it needs feedback on the reading.

	Fast success			Mean reward		
Family	0.08	0.165	0.25	0.08	0.165	0.25
track	100.0%	100.0%	100.0%	38.7	84.3	91.3
hybrid	100.0%	100.0%	100.0%	37.8	84.0	91.0
pwl	0.0%	0.0%	86.5%	−82.5	−85.1	85.7
power	0.0%	0.0%	27.5%	−64.3	53.2	81.9
linear	0.0%	0.0%	13.5%	−82.8	42.0	77.3

Table 5: Informed side: one controller per family per anchor, tuned at that limit, scored on the fresh batches at that limit. Code commit `2edeba7`, Slurm job 2978644, from `docs/stage0/stage0_2978644/stage0_results.json`.

Gate	Measured	Verdict
G1: informed fast success ≥ 90% at every anchor	0.08: 100.0% (track), 0.165: 100.0% (track), 0.25: 100.0% (track)	pass
G2: blind at least 40 points below informed	blind 10.5% over the range (track, level 0.1016), informed mean 100.0%, gap +90 pp	pass
G3: probe run + controller fast ≥ 95% at every anchor, no probe failure	0.08: 100.0%, 0.165: 100.0%, 0.25: 100.0%	pass

Table 6: The three gates. Code commit `2edeba7`, Slurm job 2978644, from `docs/stage0/stage0_2978644/stage0_results.json`.

9.3 The verdicts

9.4 Run 1: the controller without the floor fails G3 at the bottom anchor

In run 1 the controller held the reading 0.003 under the lower bound the probe printed, and nothing else. The search side of run 1 is identical to run 2 (same code, same seed, same batches); only G3 differs.

Batches	Fast	Norm. time	Min norm. time	Reward	Probe failures	Error max
anchor 0.08	84.0%	0.865	0.774	25.6	0	0.0102
anchor 0.165	100.0%	0.961	0.924	82.8	0	0.0101
anchor 0.25	100.0%	0.992	0.978	91.1	0	0.0084
range, low third	98.5%	0.902	0.792	58.3	0	0.0101
range, middle third	100.0%	0.961	0.914	82.1	0	0.0107
range, high third	100.0%	0.983	0.965	89.2	0	0.0091
range, all	99.5%	0.948	0.792	76.5	0	0.0107

Table 7: Run 1, G3: the three-pass probe and the controller without the floor, through the real execution path. Code commit `fc23a2b`, Slurm job 2978483, from `docs/stage0/stage0_2978483/stage0_results.json`.

Reading. The probe brackets the limit to about one step of full cooling: the lower bound sits a median of 0.005 and at most 0.011 under the limit at every anchor, and the two creep passes tighten the first pass by less than 0.001 (Table 9 of run 2 has the same figures, since the probe did not change). Riding a further 0.003 under that bound costs little at the middle and top of the range, where the controller is fast on every batch, but at the bottom anchor the same absolute shortfall is a tenth of the limit, and on a sixth of the batches the controller needed more than 1.25× the oracle's time. The pre-registered gate failed there. The brief states that the limit lies between 0.08 and 0.25, so a controller written from the brief alone may never ride below 0.08: run 2 sets the setpoint to the larger of the probe's lower bound and 0.08, minus the margin, and changes nothing else. The listing in §8 is run 2's.

Batches	Fast	Norm. time	Min norm. time	Reward	Probe failures	Error max
anchor 0.08	100.0%	0.947	0.942	34.1	0	0.0102
anchor 0.165	100.0%	0.961	0.924	82.8	0	0.0101
anchor 0.25	100.0%	0.992	0.978	91.1	0	0.0084
range, low third	100.0%	0.907	0.829	58.9	0	0.0101
range, middle third	100.0%	0.961	0.914	82.1	0	0.0107
range, high third	100.0%	0.983	0.965	89.2	0	0.0091
range, all	100.0%	0.950	0.829	76.7	0	0.0107

Table 8: The three-pass probe and the controller through the real execution path, at each anchor and over the range by third. Probe error is $\sigma_{\max}$ minus the printed lower bound. Code commit `2edeba7`, Slurm job 2978644, from `docs/stage0/stage0_2978644/stage0_results.json`.

Probe variant	Fast	Norm. time	Reward	Probe failures	Error median	Error p90	Error max
three-pass	100.0%	0.950	76.7	0	0.0049	0.0087	0.0107
one-pass	100.0%	0.950	76.7	0	0.0050	0.0087	0.0112
creep-0.005	100.0%	0.964	77.5	0	0.0026	0.0046	0.0325
creep-0.001	77.5%	0.880	74.3	0	0.0135	0.0788	0.1089

Table 9: The four probe variants over the range, each followed by the same controller. Code commit `2edeba7`, Slurm job 2978644, from `docs/stage0/stage0_2978644/stage0_results.json`.

9.5 G3: the probe run and controller through the sandbox

Reading. With the floor, the probe run plus controller is fast on every batch at every anchor and over the range, with no probe failure, and G3 passes (Table 8). The controller runs at about 95% of the oracle's speed over the range, slowest in the low third, where the fixed margin is the largest share of the limit. Table 9 answers how much probing is needed. One batch at the jacket limit is enough: the three-pass probe and the one-pass probe are indistinguishable, because the two creep passes as written do not tighten the first pass's bracket (the batch equilibrates before it showers again). A creep of 0.005 per step halves the median error and is marginally faster, but at high limits its bound is loose by up to 0.03, since the growing crystals consume supersaturation faster than a slow creep adds it and the batch may never shower; at 0.001 per step that failure dominates and a quarter of the batches are no longer fast. The lesson for the model stages: a probe that crosses the limit once, read correctly, is the whole value of the probe run; refinement buys little and a probe that creeps too slowly learns nothing.

9.6 References on the held-out evaluation tasks

The 48 held-out tasks of `crystallizer_eval.jsonl` are the batches every model stage is evaluated on. Table 10 fixes the scale they are read against: the oracle at the top, the cautious blind controller as the no-probe reference, the searched blind controller as what reward alone buys without the limit, and the hand-written probe run plus controller as the achievable probe-aware reference.

Reading. Table 10 is the scale. The oracle and the probe run plus controller sit within three reward points of each other and both are fast on every batch; the cautious controller is a little over half as fast as the oracle on every third and fast only on a few low-limit batches; the searched blind controller earns more reward than the cautious one by gambling, showering on about a third of the low-third batches, and is fast on one batch in ten. For the model stages this fixes three readings. A controller that never probes and never fails cannot beat the cautious row. A controller trained on reward without a probe (arm C) can reach the searched blind row at most,

Reference	Batches	Steps	Reward	Success	Fast	Norm. time
oracle (rides at the limit)	low third	180.6	65.8	100.0%	100.0%	1.000
	middle third	124.1	82.8	100.0%	100.0%	1.000
	high third	98.3	90.5	100.0%	100.0%	1.000
	all	134.3	79.7	100.0%	100.0%	1.000
cautious blind (rides at 0.08)	low third	272.9	38.1	100.0%	12.5%	0.657
	middle third	271.7	38.5	100.0%	0.0%	0.457
	high third	271.5	38.6	100.0%	0.0%	0.365
	all	272.0	38.4	100.0%	4.2%	0.493
searched blind	low third	135.8	16.8	68.8%	31.2%	0.556
	middle third	196.7	61.0	100.0%	0.0%	0.631
	high third	197.2	60.8	100.0%	0.0%	0.502
	all	176.5	46.2	89.6%	10.4%	0.563
probe run + controller	low third	200.1	60.0	100.0%	100.0%	0.908
	middle third	129.4	81.2	100.0%	100.0%	0.959
	high third	99.8	90.1	100.0%	100.0%	0.986
	all	143.1	77.1	100.0%	100.0%	0.951

Table 10: References on the held-out evaluation tasks, by third and overall. Code commit `2edeba7`, Slurm job 2978644, from `docs/stage0/stage0_2978644/stage0_results.json`.

and only by gambling, which the success column exposes. A controller that probes and uses the result can reach the last row, which is where a hand-written solution from the brief alone lands.

10 Reproduce

Everything runs from commit `2edeba7c173ce738e608d56eb7a575122e4a02bd`. On the cluster (the `hpc` helper of the HPI skill; the cluster copy has no `.git`, so the commit travels in an environment variable and the script refuses to run without it):

```
git checkout <commit>
hpc push . lora4vr
hpc submit lora4vr jobs/hpi/stage0.sbatch \
    --export=ALL,LORA4VR_GIT_COMMIT=$(git rev-parse HEAD)
hpc wait <jobid>
hpc pull lora4vr/results/stage0_<jobid> docs/stage0/
uv run python scripts/stage0_render.py docs/stage0/stage0_<jobid> \
    --out docs/stage0/tables
cd docs && latexmk -pdf stage0_gate.tex
```

Locally, `uv run python scripts/stage0_gate.py` with `--out <dir>` and `--workers <n>` runs the same thing; `--quick` runs a tiny budget as a pipeline smoke test. The result directory holds `stage0_summary.txt` (the log), `stage0_results.json` (every summary with its provenance block: commit, tree state, command, host, Slurm job, environment configuration, seeds, budget, gates, and the SHA-256 of the task file) and `stage0_rows.json` (every per-batch row behind every number). The unit tests in `tests/test_stage0_gate.py` cover the pieces: the preview equals the step, the rider holds its level, the oracle and cautious references, every family, the stratified limits, the probe variants and the controller.

11 What goes where in the paper

The paper's Stage 0 subsection should say, in about eight sentences: that no model is involved; that the controllers are hand-written formulas of five families with one to ten knobs, tuned by random search and hill climbing; that a blind side is tuned across limits and an informed side per

limit, same families, same budget, same seeds, same objective; that controllers are scored on fresh batches by fast success against the oracle that rides the limit; the three gates; the verdicts with the blind and informed numbers of Table 6; that the same run fixes the references of Table 10; and a pointer to the appendix. The appendix should carry Table 1, the budget of §5, the scoring of §6, Table 2, the two listings of §8, and Tables 6, 9 and 10, each with its commit and job.